CSAI 301 / ARTIFICIAL INTELLIGENCE

Warehouse Intelligence Lab

Search, Local Optimization, and Reinforcement Learning

Complete project report - Phases 1 and 2

MADE BY

Ammar Ahmed • Ahmed Sameh • Kareem Wael

Under the supervision of Dr Doaa

11	2	100%
Phase 1 configurations	Admissible heuristics	Final RL success

SUMMER 2026 | REPRODUCIBLE THREE-SEED EVALUATION

Executive summary

This project studies one warehouse pickup-and-delivery problem through classical search, heuristic search, local optimization, and reinforcement learning. A shared model prevents the phases from becoming unrelated demos: every method uses the same state semantics, actions, obstacles, pickup rule, delivery rule, and weighted movement costs.

The default 32 by 32 environment contains approximately 1,500 valid modeled states, satisfying the required 1,000-10,000 range. Phase 1 implements eleven required configurations. Phase 2 learns a tabular Q-policy and preserves complete midpoint and final Q-value snapshots. The animated web laboratory visualizes both phases and their evidence.

PRIMARY RESULT

UCS and both A* configurations achieved the minimum mean cost of 59.0. Manhattan A* used 308 mean expansions versus 1,152.7 for UCS. Q-learning reached 100% success over the final 100 episodes on all three seeds.

1. Problem definition and assumptions

The robot starts at S, must visit pickup P, and may complete the task only after carrying the package to delivery D. Pickup is automatic when the robot enters P.

Model element	Definition
State	(row, column, has_package). The package flag preserves the Markov property.
Actions	UP, DOWN, LEFT, RIGHT on the four-neighbor grid.
Transitions	Legal moves enter an adjacent non-wall cell. RL collisions stay in place.
Cost	Entering normal, medium, or heavy terrain costs 1, 2, or 4.
Goal	At delivery with has_package = true.
State count	Two times the traversable cells; approximately 1,500 in default worlds.

All movement costs are positive. Weighted terrain deliberately separates minimizing moves from minimizing total cost. Environment generation is seeded and rejects any map in which S-to-P or P-to-D is unreachable.

2. Software architecture

The root-level structure mirrors the assignment. environment.py owns the problem and MDP; state.py owns shared immutable types; and each algorithm has a dedicated module. search_core.py centralizes trace recording and path reconstruction. local_search_core.py centralizes action-sequence evaluation. compare.py runs reproducible experiments, visualization.py generates static evidence, and api.py exposes the same implementations to the React interface.

- One problem definition feeds search and reinforcement learning.

- Every algorithm remains readable enough for oral explanation.
- The web layer calls submitted Python code through a validated API.
- Seeded random generators make stochastic experiments reproducible.

3. Phase 1 - search algorithms

3.1 Uninformed search

Breadth-First Search uses a FIFO frontier. It is complete on this finite graph and optimal in number of moves, but not in weighted cost. Depth-First Search uses a LIFO frontier and a closed set; it is complete for this finite graph but returns the first encountered route without an optimality guarantee.

Uniform-Cost Search orders the priority queue by cumulative cost g(n). Positive movement costs make it complete and minimum-cost optimal. Iterative Deepening Search repeats depth-limited DFS. It combines depth optimality with DFS-like memory, but its repeated shallow passes are visibly expensive.

3.2 Informed search and heuristics

Greedy Best-First Search orders only by h(n), making it aggressive but non-optimal. A* orders by f(n) = g(n) + h(n). With either provided consistent admissible heuristic, A* is complete and cost-optimal.

Before pickup, each heuristic estimates current-to-P plus P-to-D. After pickup it estimates current-to-D. Manhattan distance is the sum of orthogonal coordinate differences. Euclidean distance is straight-line distance. Both ignore walls and use the minimum step cost, so neither overestimates. Manhattan is normally more informed on this four-neighbor grid.

3.3 Local search

Hill Climbing, Simulated Annealing, and the Genetic Algorithm optimize fixed-length action sequences. Simulation scores mission completion, terrain cost, collisions, and progress. A strict feasibility tier ensures every valid delivery outranks every unfinished candidate.

Hill Climbing uses mutations and random restarts. Simulated Annealing accepts selected worse moves according to exp(delta / temperature) and then cools. The Genetic Algorithm uses tournament selection, elitism, crossover, mutation, and immigrants. Finite versions of all three remain neither complete nor optimal.

4. Phase 1 experimental evaluation

The controlled experiment used weighted 32 by 32 warehouses with obstacle ratio 0.22 and seeds 7, 11, and 19. Each configuration received the same world for a given seed. Work means expanded nodes for graph search and evaluated candidates for local search.

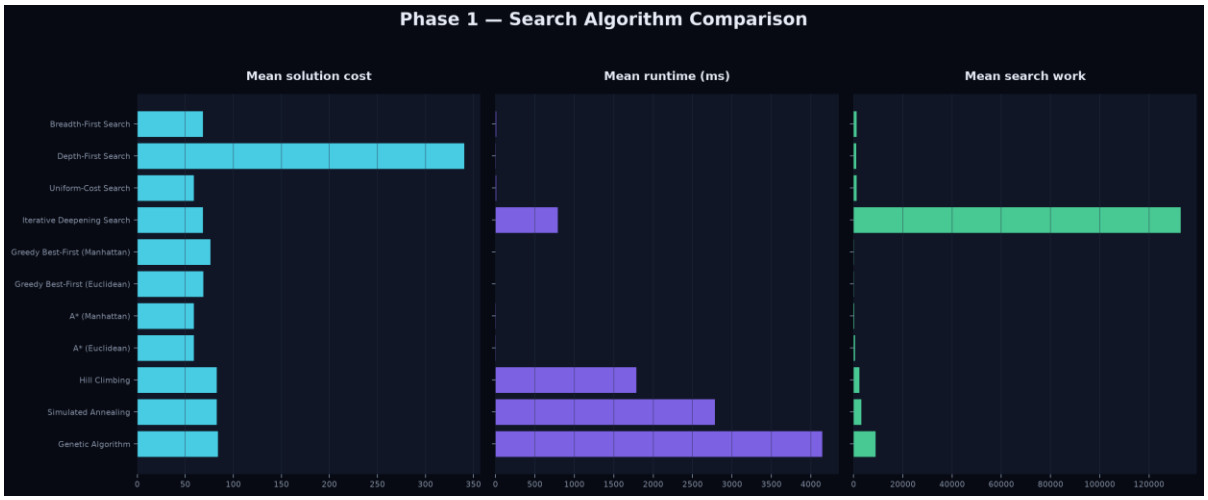

Figure 1. Mean cost, runtime, and work across the three controlled environments.

Configuration	Success	Cost	Runtime	Work	Complete	Optimal
BFS	100%	68.3	7.8	1,224	Yes	No
DFS	100%	340.3	7.0	1,019	Yes*	No
UCS	100%	59.0	8.5	1,153	Yes	Yes
IDS	100%	68.3	790.3	132,846	Yes	No
Greedy / Man.	100%	76.0	0.5	57	Yes*	No
Greedy / Euc.	100%	68.7	0.4	49	Yes*	No
A* / Man.	100%	59.0	2.4	308	Yes	Yes
A* / Euc.	100%	59.0	4.1	516	Yes	Yes
Hill Climbing	100%	82.7	1,789	2,406	No	No
Annealing	100%	82.7	2,785	3,204	No	No
Genetic	100%	84.0	4,146	8,884	No	No

Table 1. Three-seed means. Runtime is milliseconds. Yes applies to the finite closed-set implementation.*

Manhattan A* is the strongest overall classical method in this experiment. It matches UCS's optimum while using about 73% fewer expansions. Greedy is fastest but gives up cost quality. IDS confirms its memory-oriented theory but repeats very large amounts of work. Local search finds feasible routes yet spends more runtime and carries no proof of optimality.

5. Visualization and trace design

Algorithms emit renderer-independent events. A trace carries state, event type, frontier size, and available g, h, and f values. Local-search events carry the best candidate path. Trace storage is capped and reports truncation explicitly, preventing IDS or long stochastic runs from exhausting memory.

The premium web laboratory includes an animated landing page, algorithm cards, environment controls, search playback, evolving candidate routes, theoretical guarantees, selectable comparison charts, Q-

learning curves, midpoint/final Q-value cards, and a final policy overlay. Static figures come from the same experiment data.

6. Phase 2 - Q-learning

The reinforcement-learning phase retains the same state and four actions. The update is Q(s,a) <- Q(s,a) + alpha [r + gamma max Q(s',a') - Q(s,a)]. Alpha is 0.24 and gamma is 0.96. Epsilon-greedy exploration decays from 1.00 to 0.04 over 1,400 episodes.

Reward component	Purpose
Delivery bonus	Large terminal reward for completing the actual mission.
Pickup bonus	Marks completion of the first required stage.
Terrain cost	Penalizes every legal move by the entered-cell cost.
Collision	Extra penalty for attempting an illegal move.
Progress	Small signal toward P before pickup and D afterward.
Timeout	Penalty when the episode exceeds its step budget.

The complete Q-table is saved at episode 700 and at episode 1,400. Evaluation disables exploration and follows the final greedy policy. A successful evaluation must visit pickup before delivery and must respect the same transition rules as search.

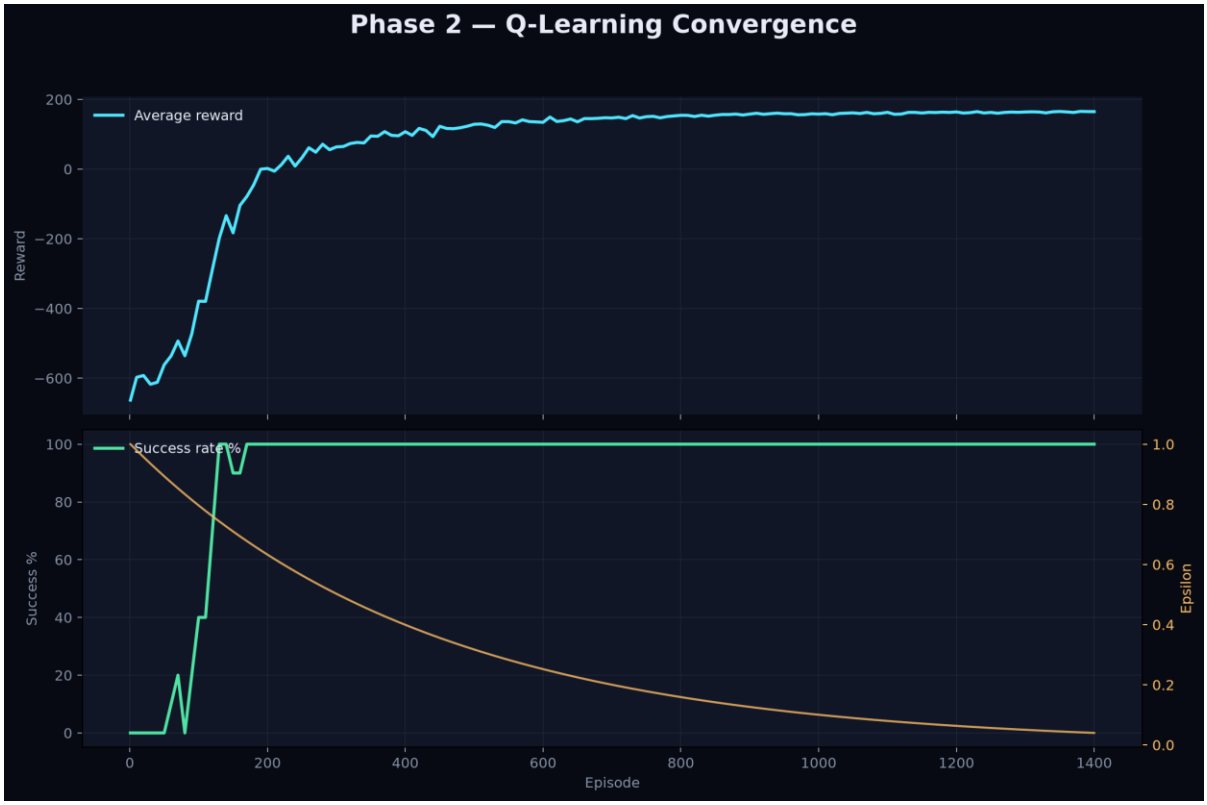

Figure 2. Reward, success rate, steps, and epsilon during seed-7 learning.

7. Phase 2 results and policy analysis

Seed	Wins	Final 100	Evaluation	Policy cost	Runtime
7	1,298/1,400	100%	Valid	72	1,655 ms
11	1,336/1,400	100%	Valid	38	1,202 ms
19	1,300/1,400	100%	Valid	73	1,685 ms

All three final policies completed the task without exploration, and every run achieved 100% success over its final 100 training episodes. Policy cost varies because each seed produces a different warehouse and Q-learning optimizes shaped discounted return rather than proving a shortest path.

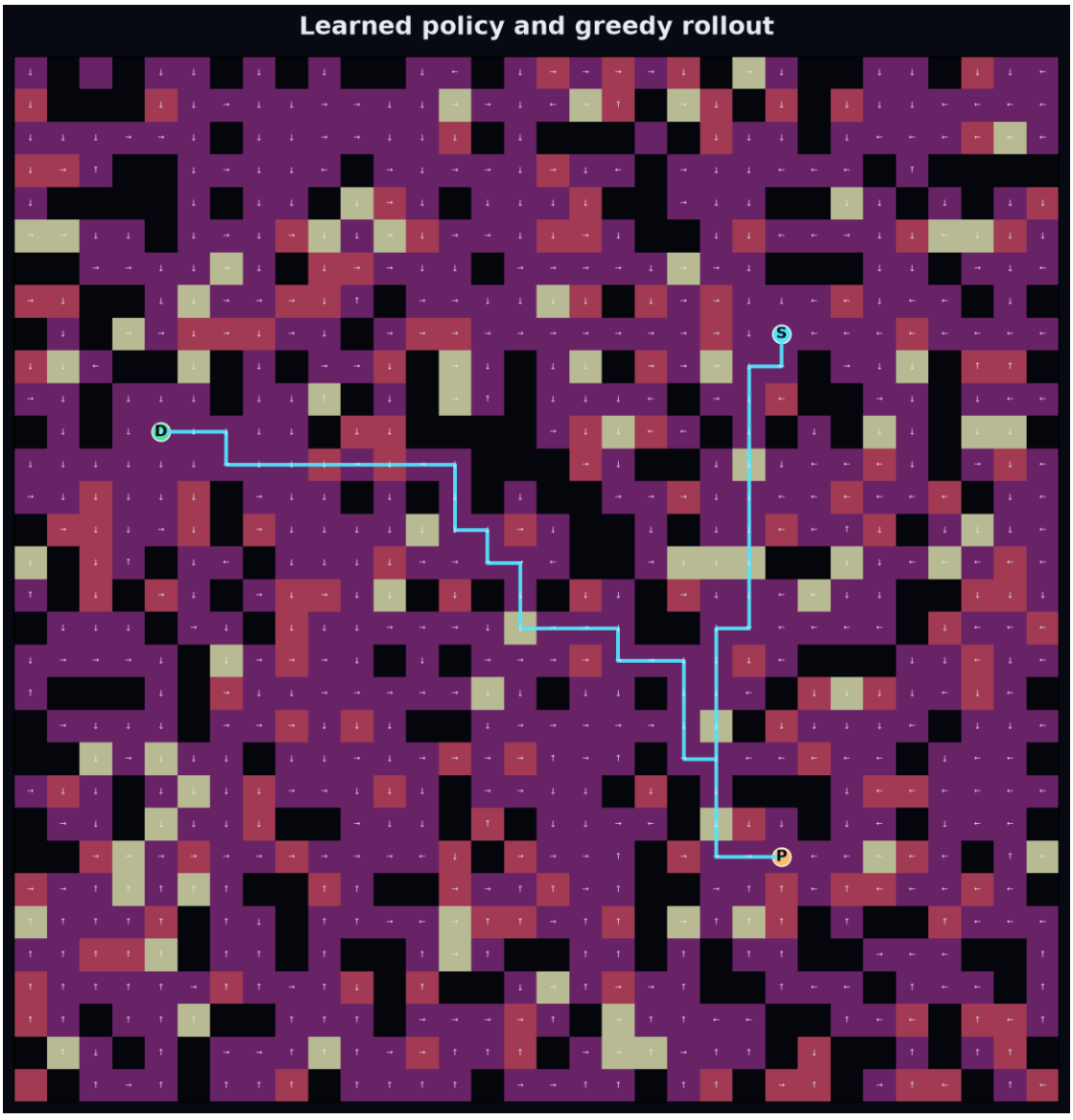

Figure 3. Greedy evaluation policy for the seed-7 warehouse.

INTERPRETATION

The learned policy is empirically reliable, not mathematically cost-optimal. Exact optimality belongs to UCS and A* under their stated assumptions.

8. Testing, reliability, and limitations

Automated tests cover environment invariants, state-space sizing, successors, reward semantics, classical optimality relationships, both heuristics, every local method, Q-value snapshots, policy evaluation, API validation, and experiment export. Ruff checks Python; Oxlint and TypeScript check the frontend; Vite creates a production build.

- Local random generators prevent global RNG interference.
- API models reject malformed grids and out-of-range values.
- The UI clears stale results after algorithm, heuristic, or environment changes.
- CSV and JSON exports preserve evidence beyond screenshots.

Limitations are explicit. Three seeds do not characterize every map distribution. Runtime depends on hardware. Tabular Q-learning scales poorly beyond assignment-sized state spaces. Local-search quality depends on finite budgets. Reward shaping and finite training do not prove that the learned policy minimizes true terrain cost.

9. Conclusion

Both phases are complete on one coherent model. The results demonstrate why algorithm guarantees matter: UCS and A* prove minimum weighted cost, BFS and IDS target depth, Greedy trades quality for speed, and local methods trade guarantees for stochastic exploration. Q-learning replaces explicit planning with learned action values and produces a reliable final policy. Source, raw data, figures, Q-tables, tests, report materials, and the interactive lab form one reproducible submission package.

Appendix A - reproducibility

Run scripts/verify.ps1 to execute all automated checks. Regenerate the complete evidence set with:

```
python compare.py --output results --seeds 7 11 19 --rows 32 --cols 32 --obstacles 0.22 --q-episodes 1400
```

Evidence files include phase1_raw.csv, phase1_summary.json, phase2_summary.json, q_values_midpoint.csv, q_values_final.csv, three PNG figures, and experiment_manifest.json.